

DATA-260 Homework 5

Pinal Pawar

GitHub repository: <https://github.com/PinalPawar/data260-7875> (collaborators Sbnikitha and supriyaselvanganesan have access)

Section 0 - Personal Configuration and Domain

Value	Definition	This submission
SID4	Last four digits of SJSU Student ID	7875
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	8675
PREFIX	"s" + SID4	s7875 (database s7875_rel)
SEED	SID4	7875
VERIFY_SEED	$260000 + \text{SID4}$	267875
DOMAIN_ID	$\text{SID4} \bmod 8$	3 - Grocery supply and recall notices
Hardware		MacBook Air, Apple M2, 16 GB RAM
Local model		qwen3:1.7b via Ollama
Tagged commit		d3fe5d294b3d1ba2b7a65eed3d02e02fdbd00b65 (code and data; the hw5 tag adds this report)

The self-check script `scripts/verify_hw05.py` passed 17 of 17 checks on my machine and wrote `reports/hw05/verification.json`.

```

((data260) Pinal Pawar data260-7875-main % python3 scripts/verify_hw05.py 2>/dev/
null | tee -a reports/hw05/RUN_LOG.txt
HW5 smoke test -- Pinal Pawar (SID4 7875) -- 2026-10-02T01:41:57
[PASS] backend responds on PORT_BASE -- port 8675, HTTP 401
[PASS] anonymous request is refused -- HTTP 401
[PASS] login works -- HTTP 200
[PASS] manufacturers list is paginated -- total=10
[PASS] relationship endpoint returns notices -- HTTP 200
[PASS] unknown manufacturer returns 404 -- HTTP 404
[PASS] meals MCP server starts with 4 tools -- search_meals_by_name, meals_by_in
gredient, meal_details, random_meal
[PASS] meals MCP server answers a tool call -- search_meals_by_name
[PASS] domain MCP server starts with exactly 3 tools -- search_notices, get_noti
ce_detail, recall_stats
[PASS] domain tool call returns the {ok, data, error} envelope -- ok=True
[PASS] execute_tool rejects an invalid input without raising
[PASS] execute_tool enforces the safety rule
[PASS] offline test suite passes -- 10/10 tests passed
[PASS] 150 fault-injection call records (3 rates x 50) -- 150 rows
[PASS] agent log has at least 4 runs with a stop reason -- 6 runs: ['completed',
'max_steps', 'safety_block']
[PASS] raw/mcp_inspector_meals_server.json present
[PASS] raw/mcp_inspector_recall_server.json present

17/17 checks passed -> reports/hw05/verification.json
((data260) Pinal Pawar data260-7875-main %

```

Figure 1. Smoke test output: backend, both MCP servers, tool layer and raw files.

Part 1 - Second Entity, API and Redux Client

I. Database

My primary entity is the recall notice. The related entity is the manufacturer: one manufacturer issues many notices. Before this homework the manufacturer was a free-text column on each notice, so I moved it into its own table and linked the two with a foreign key.

Table	Fields
manufacturers	id (PK), name (primary text), headquarters (secondary text), code (unique, format MFR-0001), created_at, updated_at
notices	id (PK), product (primary field), notice_code (unique, format RCL-2026-00001), units_affected (integer, default 0), manufacturer_id (FK to manufacturers.id), email, description, category, created_at, updated_at

```
notice_code      = Column(String(20), nullable=False, unique=True, index=True)
units_affected   = Column(Integer, nullable=False, default=0, server_default="0")
manufacturer_id  = Column(Integer, ForeignKey("manufacturers.id", ondelete="RESTRICT"),
                          nullable=False, index=True)
```

Deletion rule. I did not implement cascade. The foreign key is ON DELETE RESTRICT, and the API checks first and returns 409, so a manufacturer that still has notices cannot be deleted.

Passwords are stored only as bcrypt hashes in users.password_hash (unchanged from HW4).

The existing HW4 table already held 5,001 notices, so I upgraded it in place with `scripts/migrate_hw05.py` instead of recreating it. No rows were lost.

```
[(data260) Pinal Pawar data260-7875-main % python3 scripts/migrate_hw05.py
[1/5] manufacturers table present
[2/5] 10 manufacturer name(s) copied into manufacturers
[3/5] new notices columns added and filled
[4/5] UNIQUE(notice_code) and FOREIGN KEY(manufacturer_id) added
[5/5] old free-text notices.manufacturer column dropped
Done: 10 manufacturers, 5001 notices.
(data260) Pinal Pawar data260-7875-main %
```

Figure 2. Migration run: manufacturers created, notices columns added, constraints applied.

```

=== manufacturers: columns ===
Field | Type | Null | Key | Default | Extra
id | int | NO | PRI | None | auto_increment
name | varchar(255) | NO | MUL | None |
headquarters | varchar(255) | NO | | None |
code | varchar(16) | NO | UNI | None |
created_at | datetime | NO | | now() | DEFAULT_GENERATED
updated_at | datetime | NO | | now() | DEFAULT_GENERATED

=== notices: columns ===
Field | Type | Null | Key | Default | Extra
id | int | NO | PRI | None | auto_increment
product | varchar(255) | NO | MUL | None |
email | varchar(255) | NO | | None |
description | text | NO | | None |
category | varchar(64) | NO | | None |
notice_code | varchar(20) | NO | UNI | None |
units_affected | int | NO | | 0 |
manufacturer_id | int | NO | MUL | None |
created_at | datetime | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED
updated_at | datetime | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP

=== manufacturers: all rows ===
id | name | headquarters | code | created_at
1 | ACME Foods | Fresno, CA | MFR-0001 | 2026-10-01 22:22:25
2 | BlueRiver Foods | Des Moines, IA | MFR-0002 | 2026-10-01 22:22:25
3 | Coastal Provisions | Wichita, KS | MFR-0003 | 2026-10-01 22:22:25
4 | Golden Acres | Boise, ID | MFR-0004 | 2026-10-01 22:22:25
5 | GreenFields Co | Salinas, CA | MFR-0005 | 2026-10-01 22:22:25
6 | Harvest Point | Des Moines, IA | MFR-0006 | 2026-10-01 22:22:25
7 | Northgate Dairy | Yuma, AZ | MFR-0007 | 2026-10-01 22:22:25
8 | Prairie Mills | Salinas, CA | MFR-0008 | 2026-10-01 22:22:25
9 | Sunny Farms Co | Fresno, CA | MFR-0009 | 2026-10-01 22:22:25
10 | Valley Fresh | Boise, ID | MFR-0010 | 2026-10-01 22:22:25

=== notices: first 5 rows ===
id | product | notice_code | units_affected | manufacturer_id
1 | Frozen Spinach (10oz) | RCL-2026-00001 | 2919 | 5
2 | Ground Beef (batch 2) | RCL-2026-00002 | 838 | 4
3 | Peanut Butter (batch 3) | RCL-2026-00003 | 3757 | 10
4 | Ice Cream (batch 4) | RCL-2026-00004 | 1676 | 3
5 | Deli Turkey (batch 5) | RCL-2026-00005 | 4595 | 10

=== notices per manufacturer ===
code | name | notices
MFR-0001 | ACME Foods | 541
MFR-0002 | BlueRiver Foods | 488
MFR-0003 | Coastal Provisions | 510
MFR-0004 | Golden Acres | 471
MFR-0005 | GreenFields Co | 478
MFR-0006 | Harvest Point | 513
MFR-0007 | Northgate Dairy | 495
MFR-0008 | Prairie Mills | 523
MFR-0009 | Sunny Farms Co | 502
MFR-0010 | Valley Fresh | 480
(data260) Pinal Pawar data260-7875-main %

```

Figure 3. Database after migration: column definitions (PRI, UNI, MUL keys), the 10 manufacturers, sample notices, and notices per manufacturer.

II. API Implementation

Both entities have full CRUD, validated with Pydantic schemas. All endpoints require the HW4 login session.

Entity	Endpoints
--------	-----------

Manufacturer	POST /api/manufacturers; GET /api/manufacturers?page=&page_size=(paginated); GET, PUT, DELETE /api/manufacturers/{id}
Notice	POST /api/notices; GET /api/notices?skip=&limit=; GET, PUT, DELETE /api/notices/{id}
Relationship	GET /api/manufacturers/{id}/notices - all notices for one manufacturer
Errors	404 not found; 422 validation (including the unique field's format); 409 constraint violation (duplicate code, delete blocked)

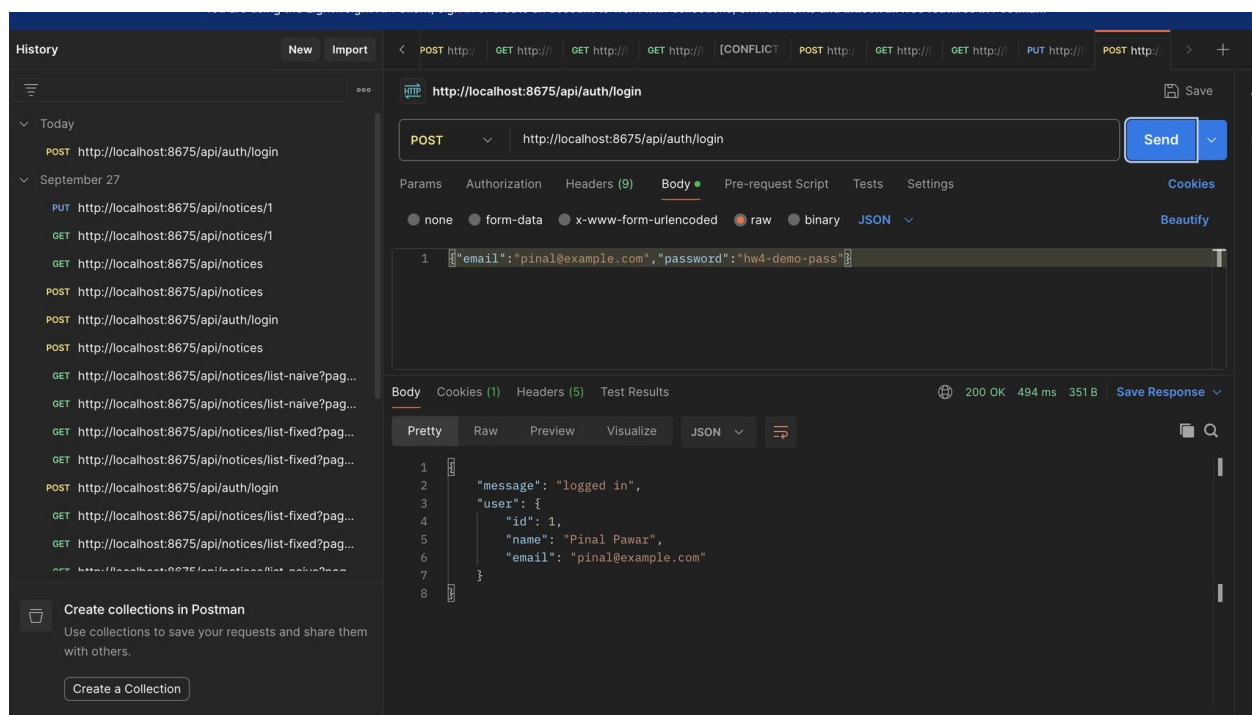


Figure 4. Login (200). Every later request uses this session cookie.

```
@router.post("", response_model=schemas.ManufacturerOut, status_code=201)
def create_manufacturer(payload: schemas.ManufacturerCreate, ...):
    try:
        return crud.create_manufacturer(db_session_based26, payload)
    except crud.Conflict as exc:
        # duplicate code
        raise HTTPException(status_code=409, detail=str(exc))
```

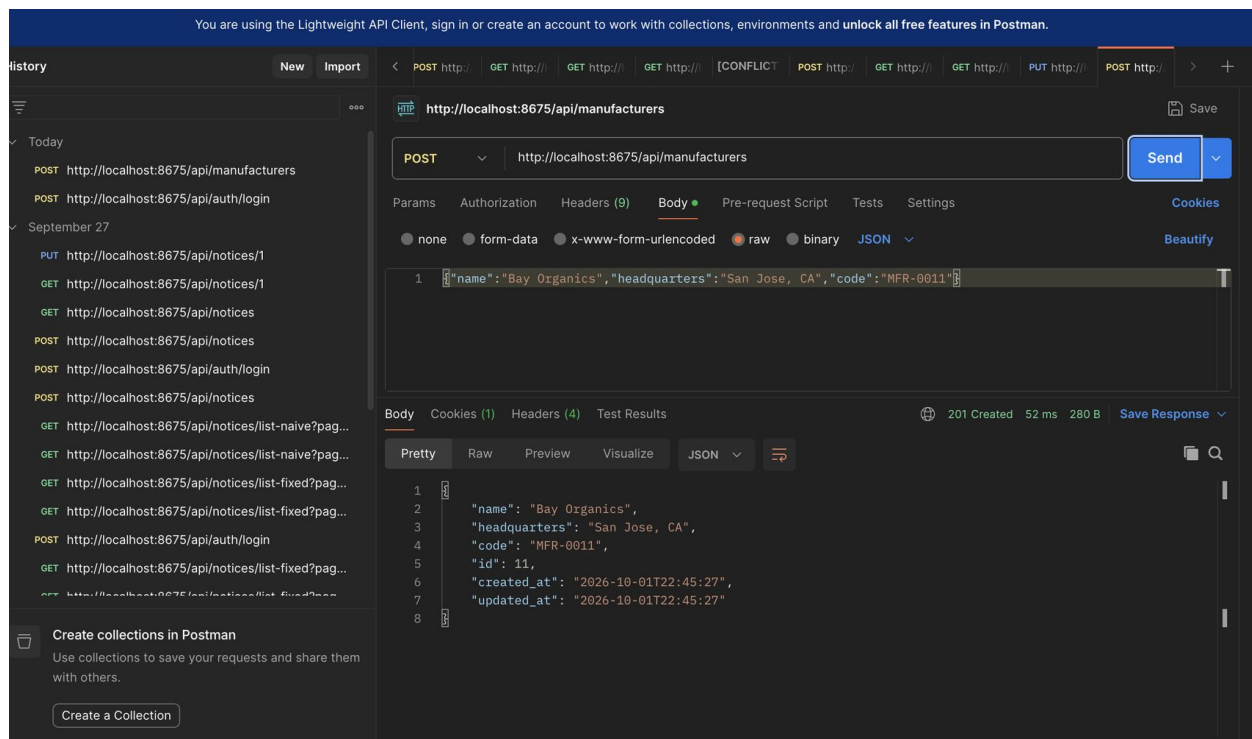


Figure 5. Manufacturer - create (201).

```
@router.get("", response_model=schemas.ManufacturerPage)
def list_manufacturers(page: int = Query(1, ge=1), page_size: int = Query(10, ge=1,
le=100), ...):
    items, total = crud.list_manufacturers(db_session_baseded26, page, page_size)
    return {"items": items, "total": total, "page": page, "page_size": page_size}
```

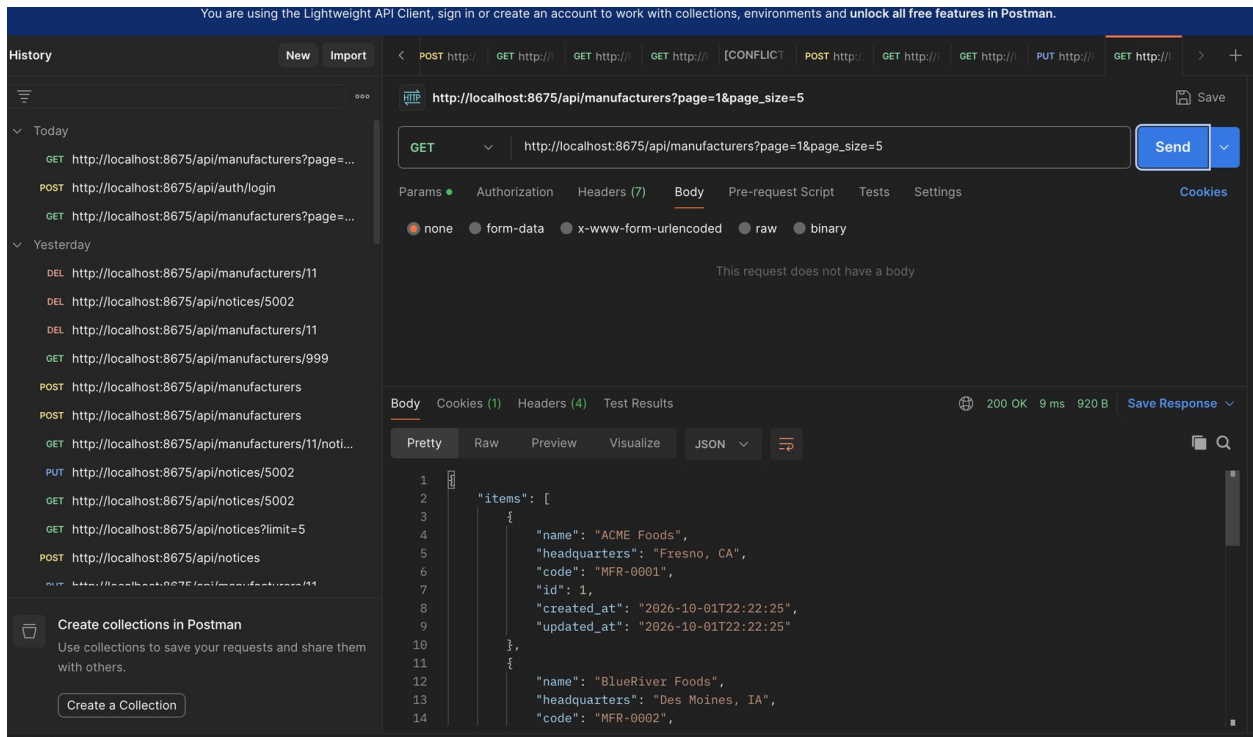


Figure 6. Manufacturer - list with pagination, page=1 and page_size=5 (200).

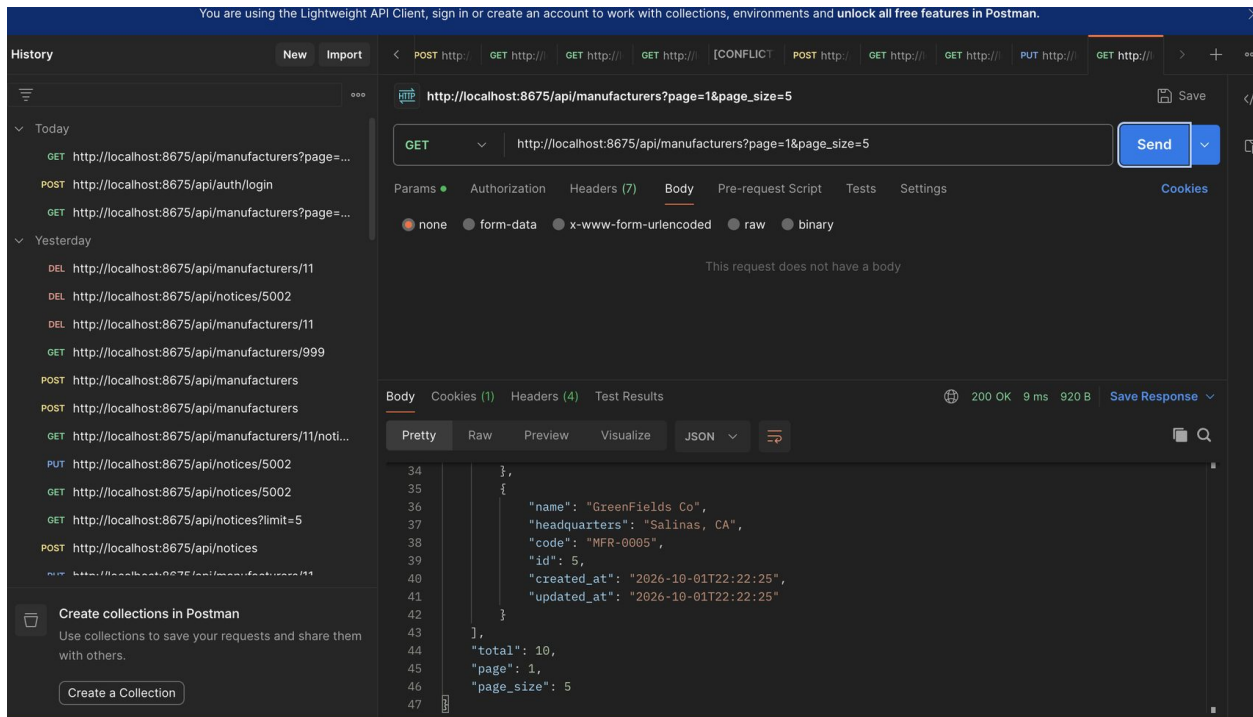


Figure 7. Same response scrolled to the end: total, page and page_size.

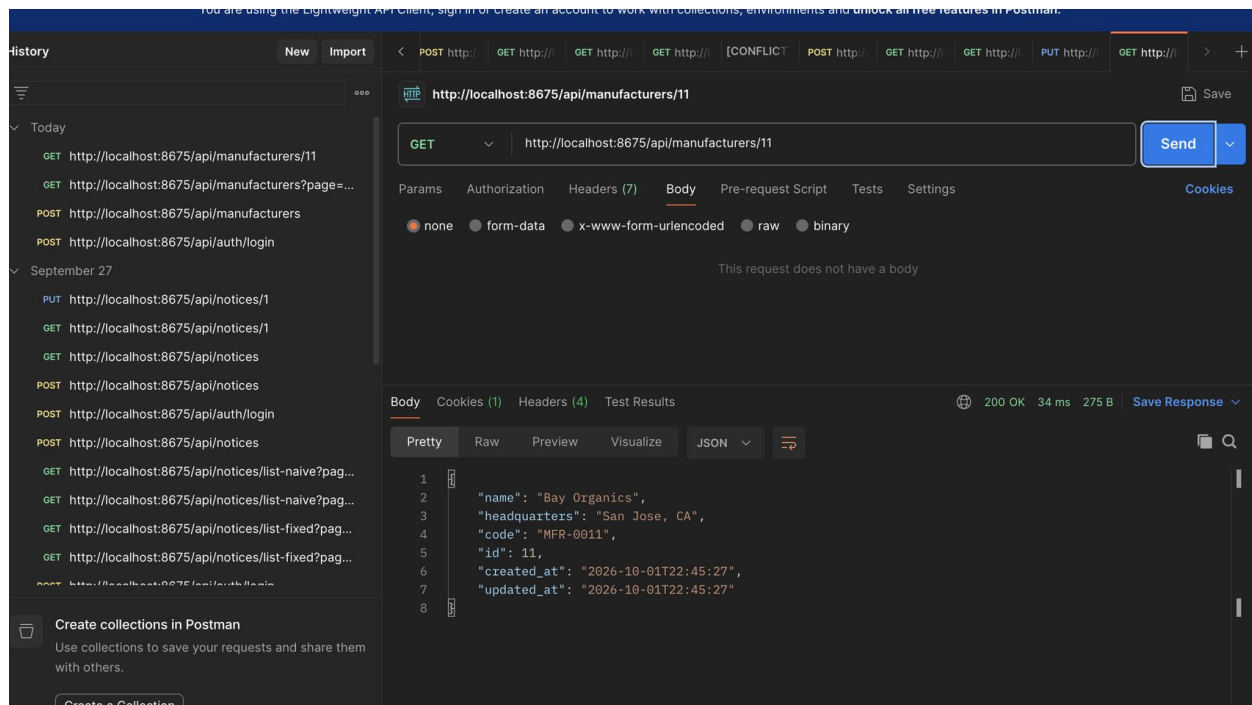


Figure 8. Manufacturer - get one record (200).

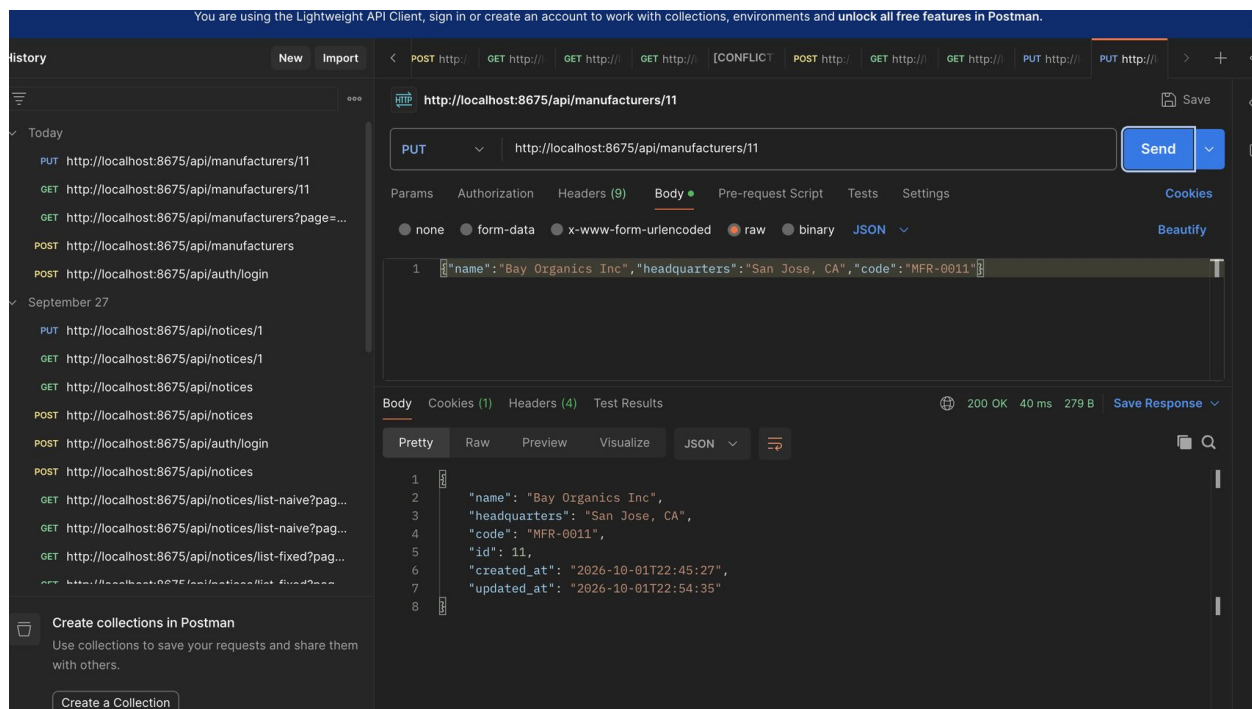


Figure 9. Manufacturer - update (200); updated_at changes.

```

class NoticeBase(BaseModel):
    product: str = Field(min_length=1, max_length=255)
    notice_code: str = Field(pattern=r"^\RCL-\d{4}-\d{5}$")
    units_affected: int = Field(default=0, ge=0)
    manufacturer_id: int = Field(gt=0)

```

```
@app.post("/api/notices", response_model=schemas.NoticeOut, status_code=201)
def create_notice(notice: schemas.NoticeCreate, ...):
    try:
        return crud.create_notice(db_session_base26, notice)
    except crud.NotFound as exc:
        raise HTTPException(status_code=404, detail=str(exc))
    except crud.Conflict as exc:
        raise HTTPException(status_code=409, detail=str(exc))
```

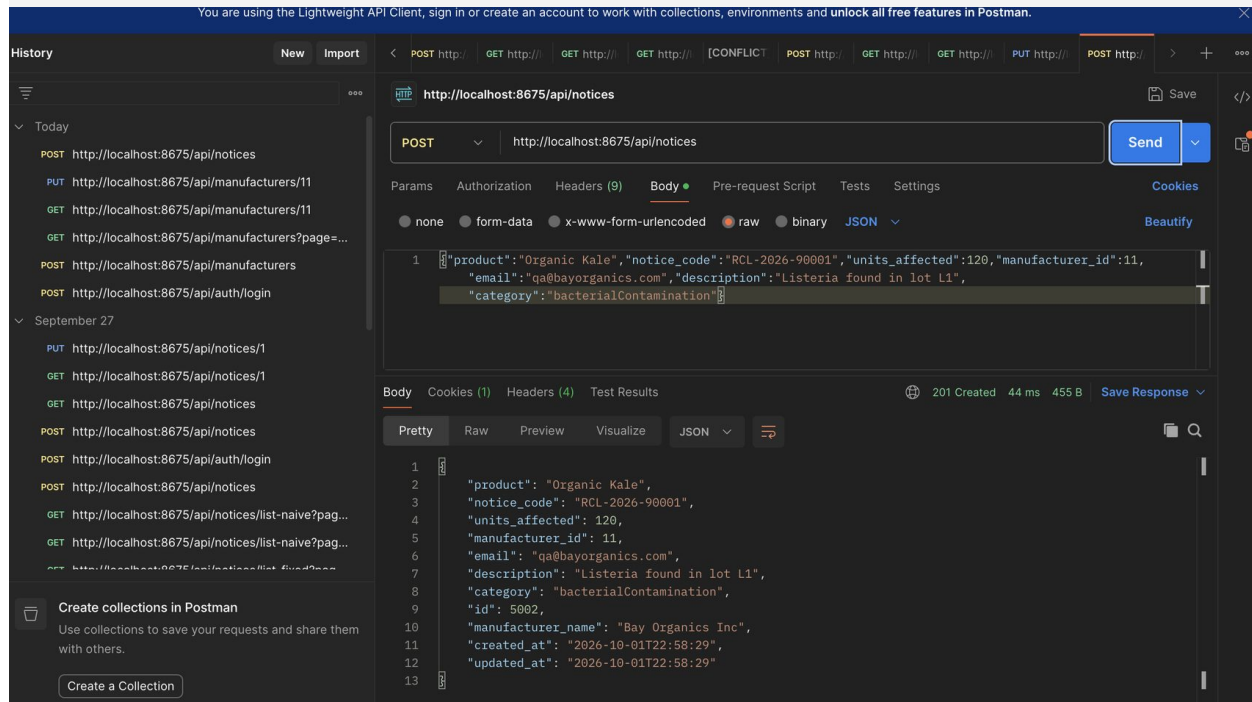


Figure 10. Notice - create (201), linked to manufacturer 11.

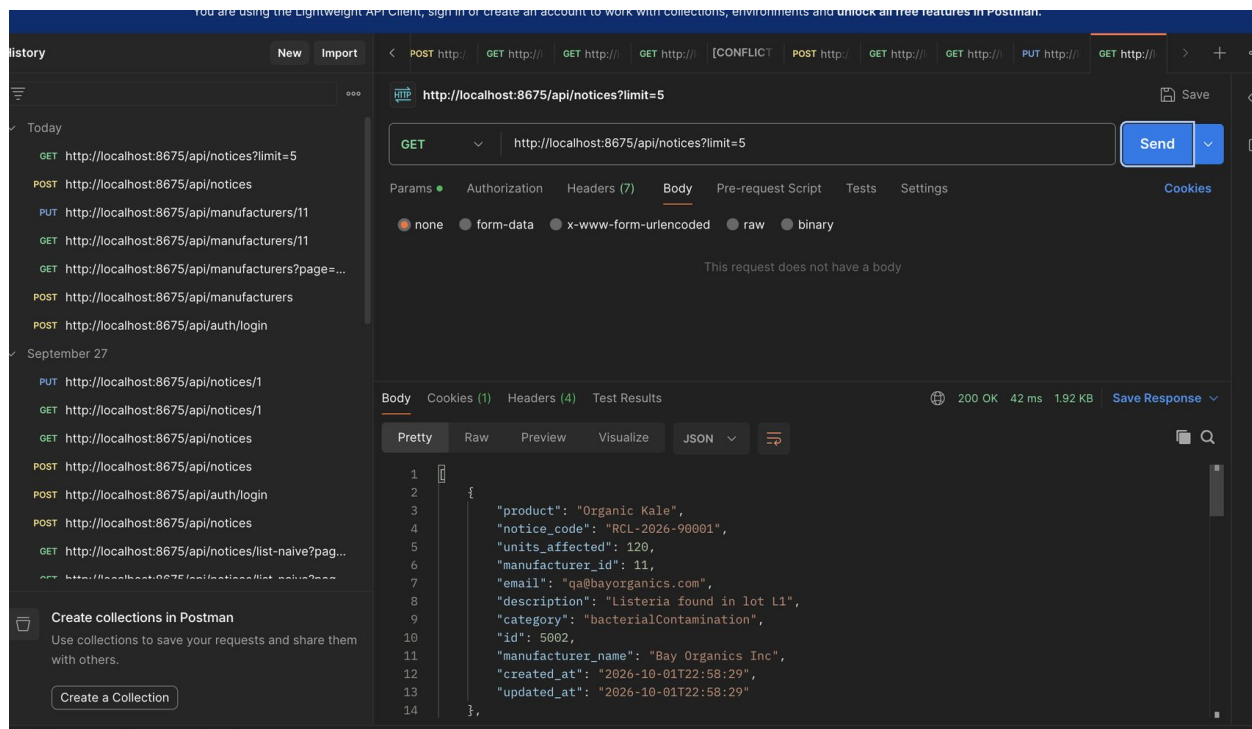


Figure 11. Notice - list (200).

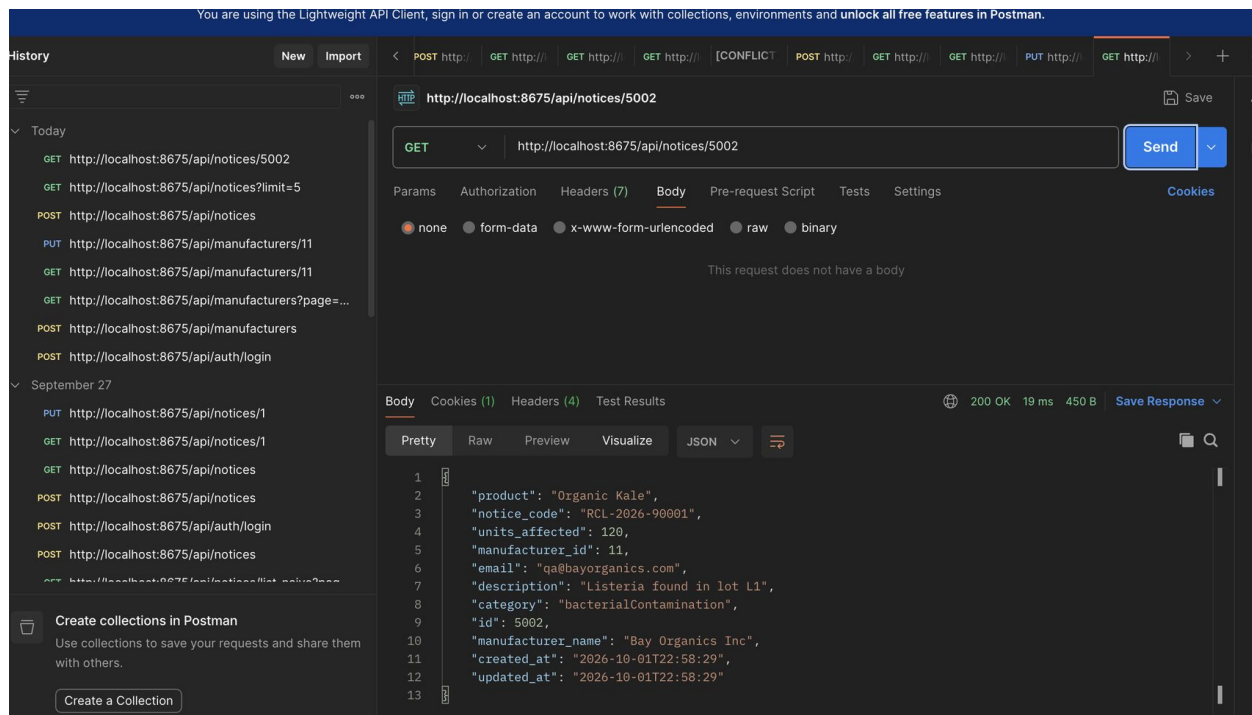


Figure 12. Notice - get one record (200).

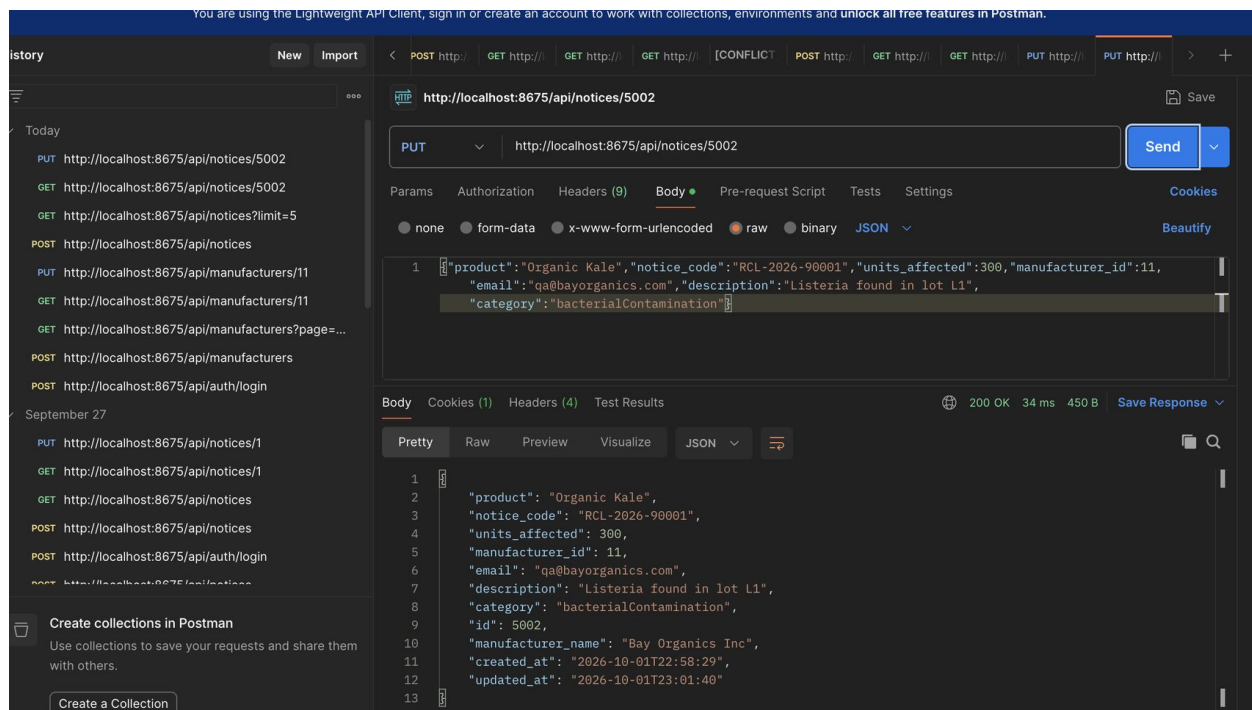


Figure 13. Notice - update (200); units_affected 120 to 300.

```

@router.get("/{manufacturer_id}/notices", response_model=List[schemas.NoticeOut])
def notices_for_manufacturer(manufacturer_id: int, skip: int = 0, limit: int = 50, ...):
    return crud.list_notices_for_manufacturer(db_session_based26, manufacturer_id, skip,
limit)

```

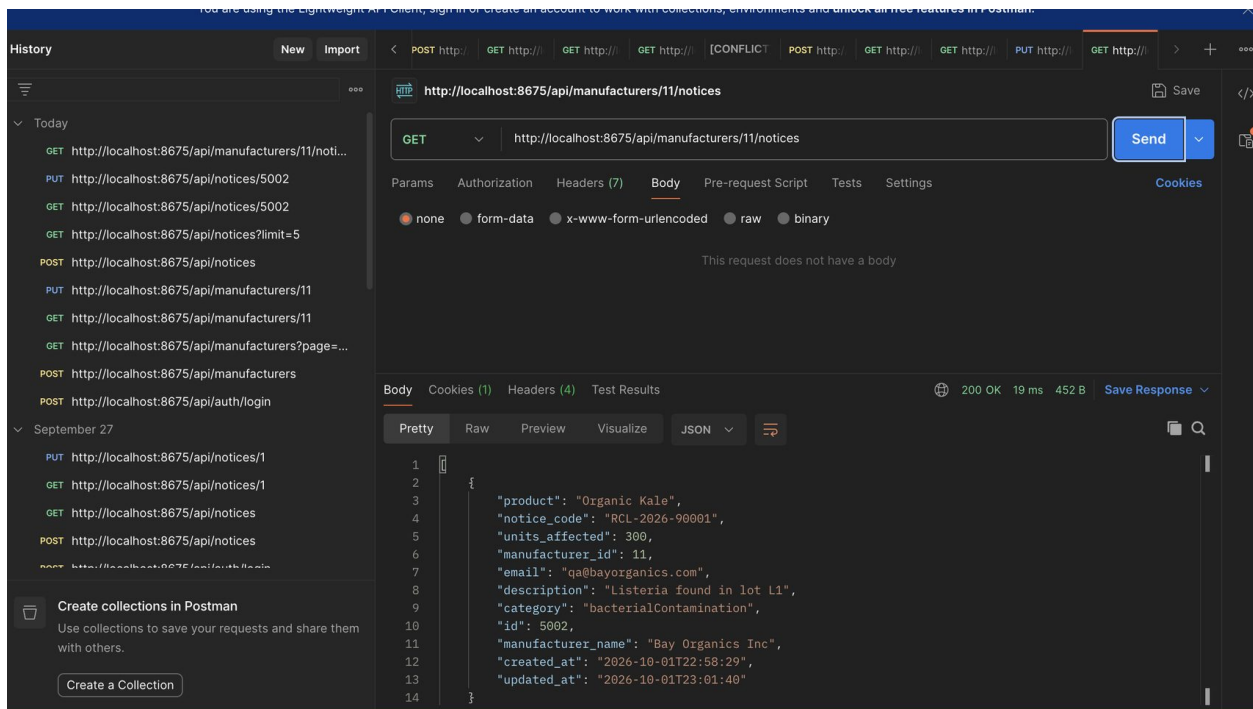


Figure 14. Relationship query - notices for manufacturer 11 (200).

```

class ManufacturerBase(BaseModel):
    name: str = Field(min_length=1, max_length=255)
    headquarters: str = Field(min_length=1, max_length=255)
    code: str = Field(pattern=r"^MFR-\d{4}$") # unique field, format validated ->
422

```

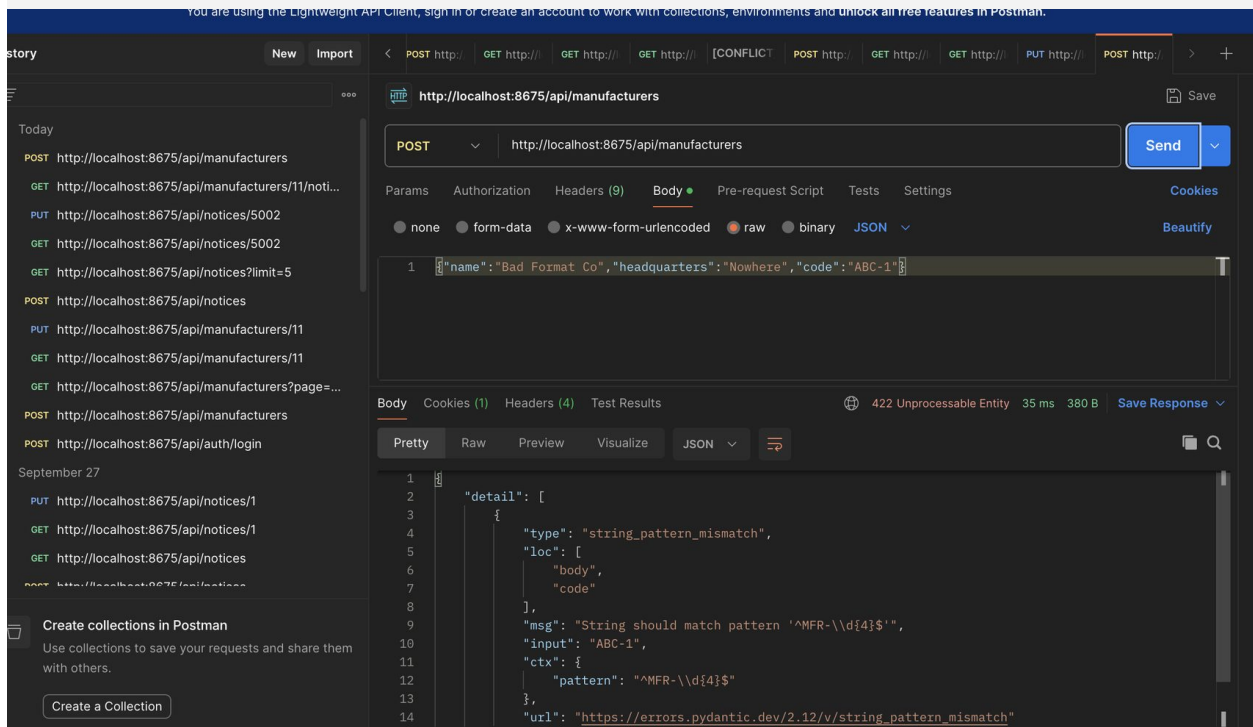


Figure 15. Validation error - code ABC-1 does not match MFR-0000 (422).

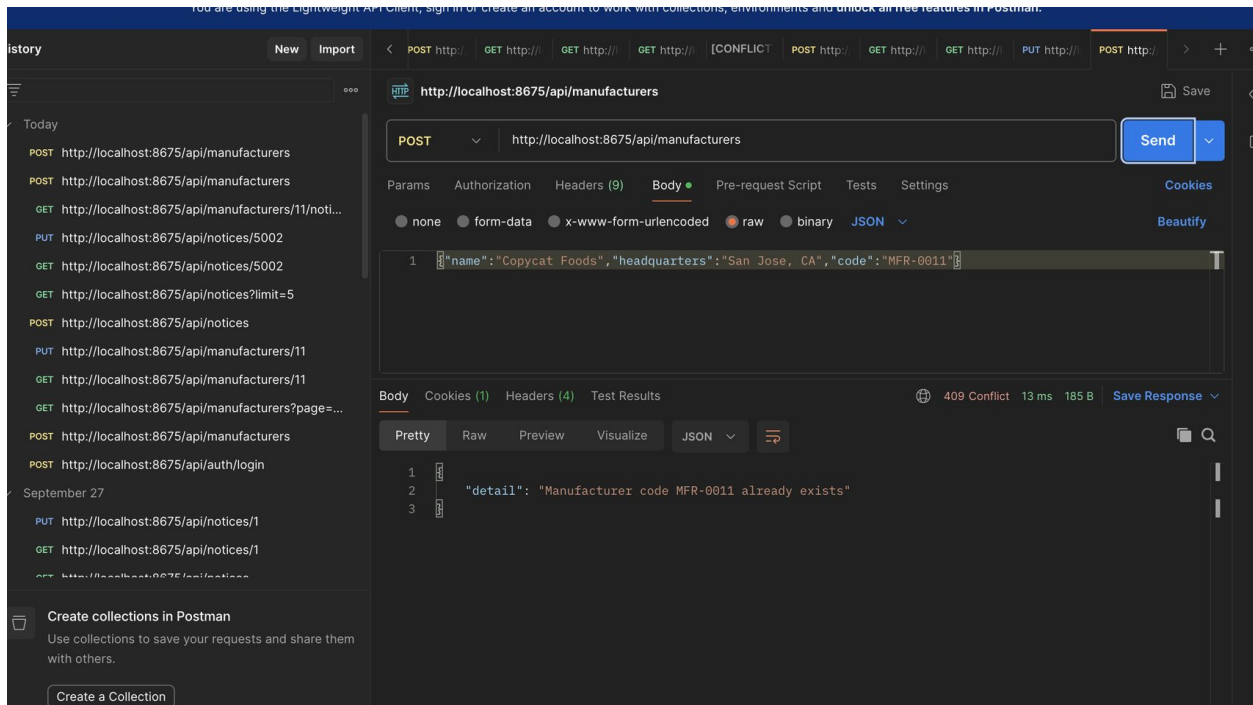


Figure 16. Constraint violation - duplicate code MFR-0011 (409).

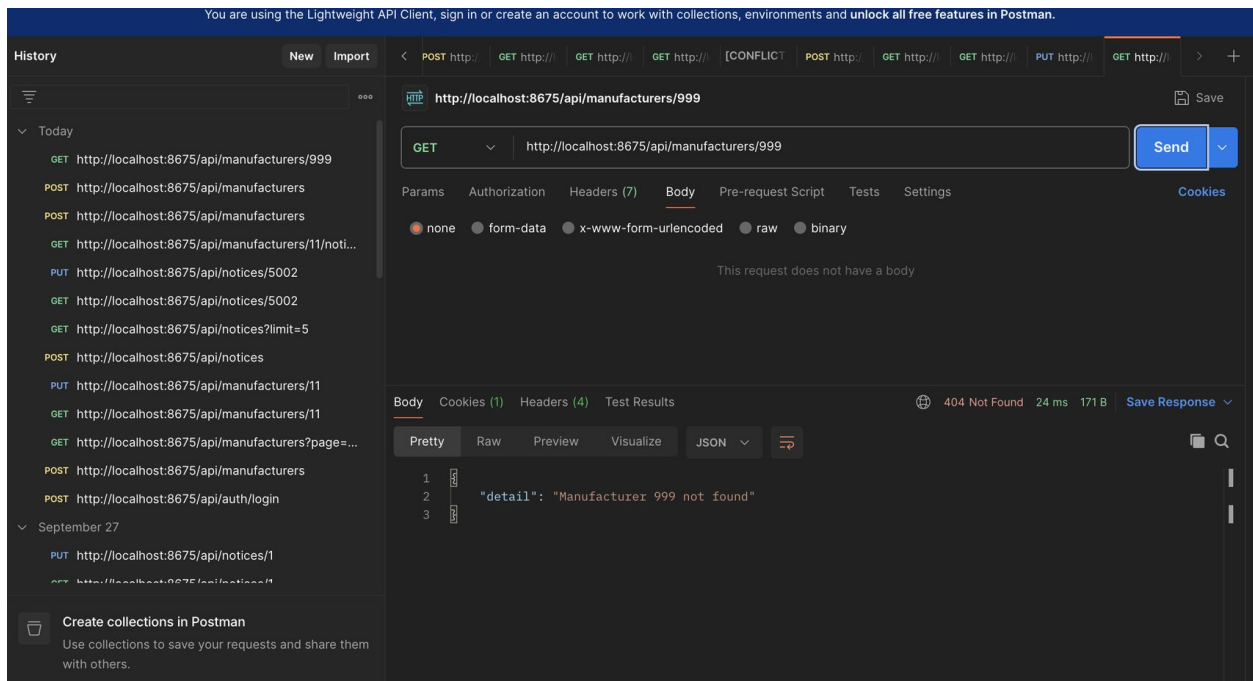


Figure 17. Resource not found - manufacturer 999 (404).

```
def delete_manufacturer(db_session_based26, manufacturer_id):
    row = get_manufacturer(db_session_based26, manufacturer_id) # raises NotFound ->
    404
    count = db_session_based26.query(func.count(Notice.id)) \
        .filter(Notice.manufacturer_id == manufacturer_id).scalar()
    if count:
```

```
raise Conflict(f"Cannot delete manufacturer {manufacturer_id}: it still has {count} notice(s) ")
```

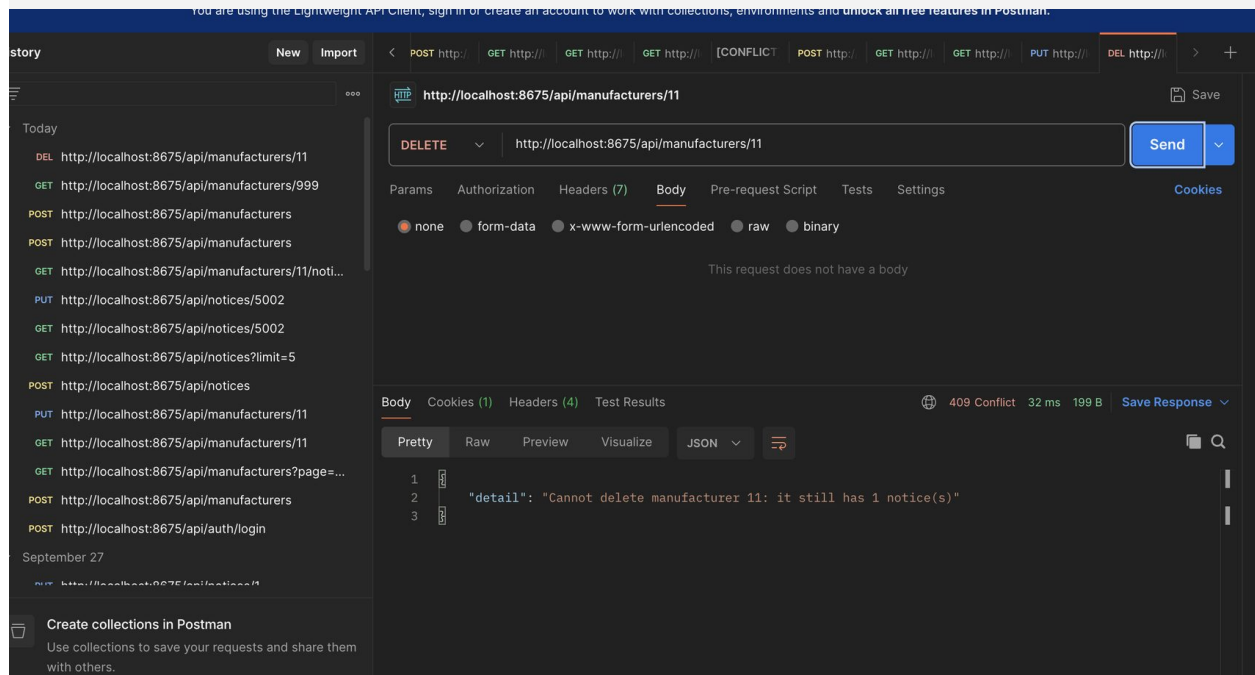


Figure 18. Delete blocked - manufacturer 11 still has a notice (409).

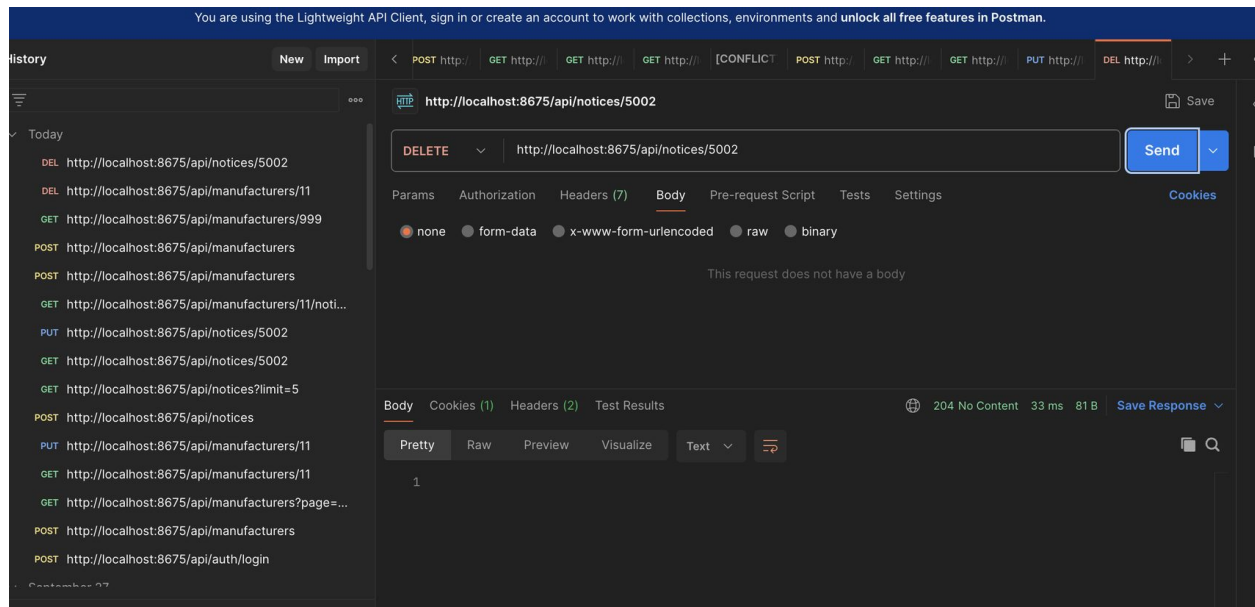


Figure 19. Notice - delete (204).

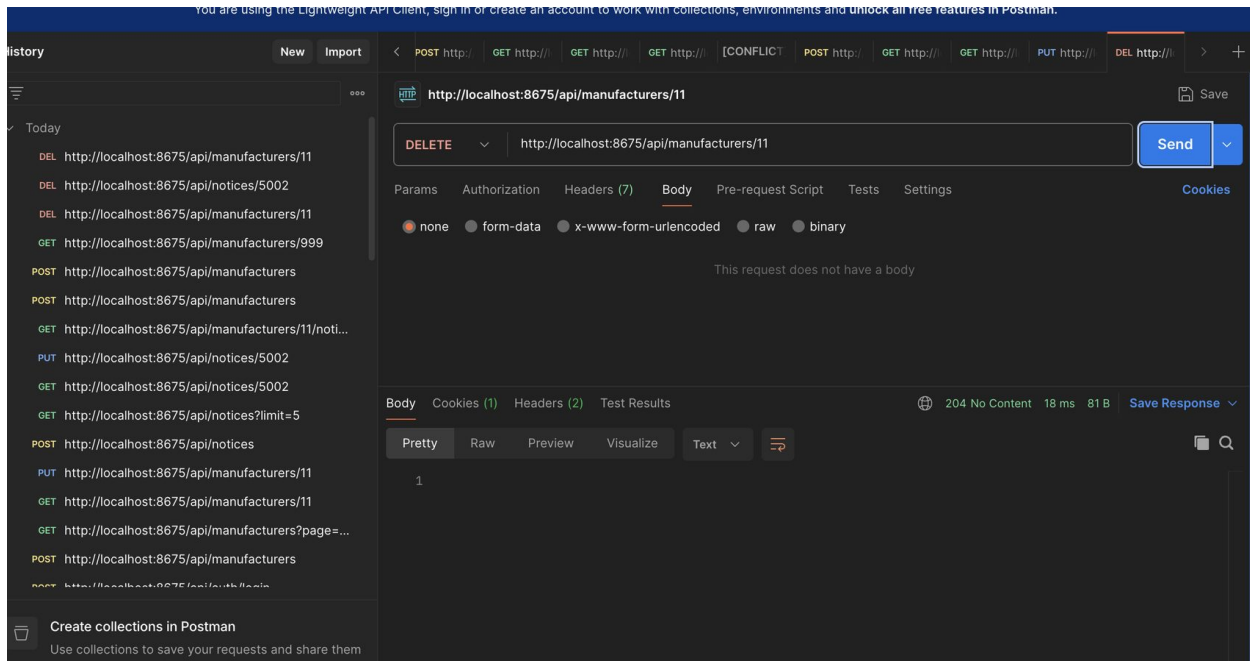


Figure 20. Manufacturer - delete succeeds once it has no notices (204).

III. Redux Client

The React app now keeps notices in a Redux Toolkit store (`store/store.js`) with one slice (`features/notices/noticesSlice.js`). Four async thunks call the API with Axios; the slice stores the list, a loading flag and the last error. Each screenshot shows the thunk and its reducer next to the working screen.

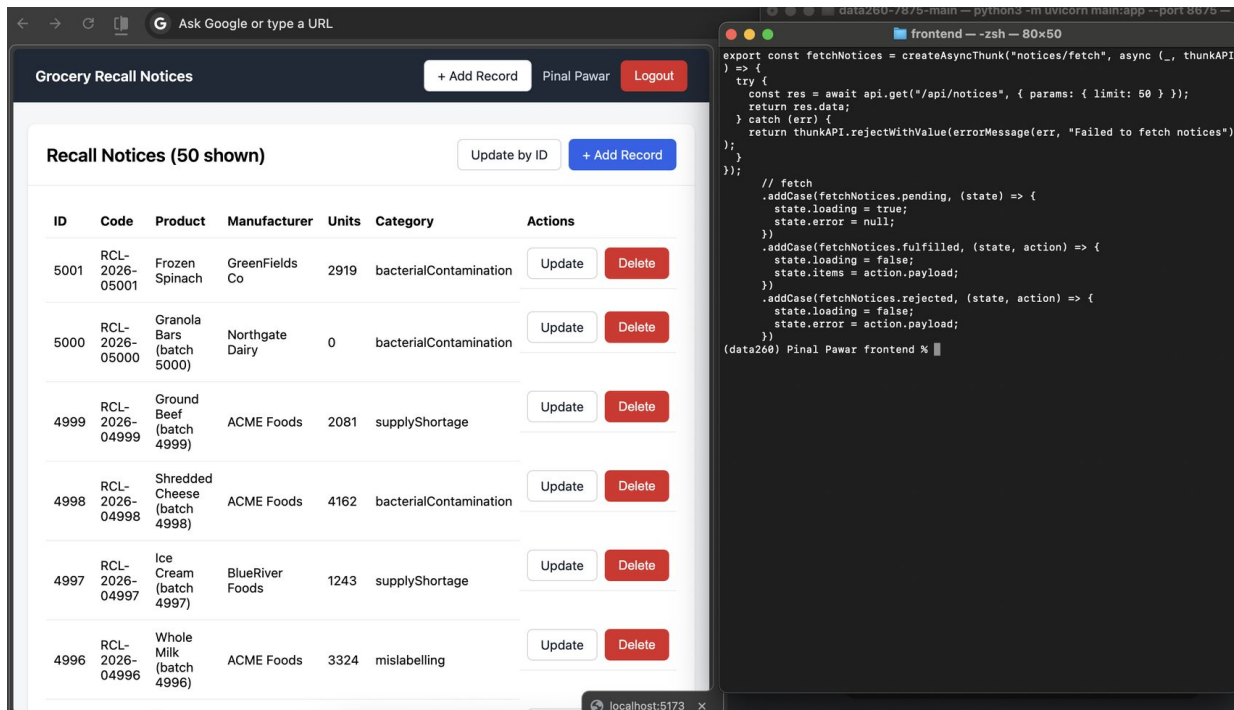


Figure 21. Home - fetchNotices thunk; the list of 50 is read from Redux state.

Report a Recall

Product (primary field)
Organic Kale

Notice code (unique, format RCL-2026-00001)
RCL-2026-90002

Units affected
120

Manufacturer
MFR-0005 - GreenFields Co

Contact email
qa@test.com

Description
Listeria found in lot L1

Category
supplyShortage

Submit Recall Notice

Figure 22. Create - the form, with the manufacturer chosen from a dropdown.

Recall Notices (51 shown)

ID	Code	Product	Manufacturer	Units	Category	Actions
5003	RCL-2026-90001	Organic Kale	BlueRiver Foods	120	supplyShortage	<button>Update</button> <button>Delete</button>
5001	RCL-2026-05001	Frozen Spinach	GreenFields Co	2919	bacterialContamination	<button>Update</button> <button>Delete</button>
5000	RCL-2026-05000	Granola Bars (batch 5000)	Northgate Dairy	0	bacterialContamination	<button>Update</button> <button>Delete</button>
4999	RCL-2026-04999	Ground Beef (batch 4999)	ACME Foods	2081	supplyShortage	<button>Update</button> <button>Delete</button>
4998	RCL-2026-04998	Shredded Cheese (batch 4998)	ACME Foods	4162	bacterialContamination	<button>Update</button> <button>Delete</button>
4997	RCL-2026-04997	Ice Cream (batch 4997)	BlueRiver Foods	1243	supplyShortage	<button>Update</button> <button>Delete</button>
...	...	...	...	...	...	...

```

export const createNotice = createAsyncThunk("notices/create", async (payload, thunkAPI) => {
  try {
    const res = await api.post("/api/notices", payload);
    return res.data;
  } catch (err) {
    return thunkAPI.rejectWithValue(errorMessage(err, "Failed to create notice"));
  }
});

// add (newest first, same order the API returns)
.addCase(createNotice.fulfilled, (state, action) => {
  state.items.unshift(action.payload);
  state.error = null;
})
.addCase(createNotice.rejected, (state, action) => {
  state.error = action.payload;
})
)(data260) Pinal Pawar frontend %
  
```

Figure 23. Create - createNotice thunk; the new record (5003) appears first on Home.

localhost:5173/update

Update Recall Notice

Notice ID
5001

Load

Editing notice **5001** (RCL-2026-05001)

Product (primary field)
Frozen Spinach

Notice code (unique, format RCL-2026-00001)
RCL-2026-05001

Units affected
2919

Manufacturer
MFR-0005 - GreenFields Co

Contact email
qa@greenfields.com

Description
Possible Listeria contamination in lot GF-2291.

Category
bacterialContamination

Save Changes

Figure 24. Update - select by ID: 5001 typed, Load pressed, form filled.

localhost:5173

Grocery Recall Notices

+ Add Record Pinal Pawar Logout

Recall Notices (51 shown)

Update by ID + Add Record

ID	Code	Product	Manufacturer	Units	Category	Actions
5003	RCL-2026-90001	Organic Kale	BlueRiver Foods	300	supplyShortage	Update Delete
5001	RCL-2026-05001	Frozen Spinach	GreenFields Co	2919	bacterialContamination	Update Delete
5000	RCL-2026-05000	Granola Bars (batch 5000)	Northgate Dairy	0	bacterialContamination	Update Delete
4999	RCL-2026-04999	Ground Beef (batch 4999)	ACME Foods	2081	supplyShortage	Update Delete
4998	RCL-2026-04998	Shredded Cheese (batch 4998)	ACME Foods	4162	bacterialContamination	Update Delete
4997	RCL-2026-	Ice Cream (batch	BlueRiver Foods	1243	supplyShortage	Update Delete

```

export const updateNotice = createAsyncThunk("notices/update", async ({ id, payload }, thunkAPI) => {
  try {
    const res = await api.put(`/api/notices/${id}`, payload);
    return res.data;
  } catch (err) {
    return thunkAPI.rejectWithValue(errorMessage(err, "Failed to update notice"));
  }
});

// update
.addCase(updateNotice.fulfilled, (state, action) => {
  const index = state.items.findIndex((n) => n.id === action.payload.id);
  if (index !== -1) state.items[index] = action.payload;
  state.error = null;
})
.addCase(updateNotice.rejected, (state, action) => {
  state.error = action.payload;
})
)
(data268) Pinal Pawar frontend %

```

Figure 25. Update - updateNotice thunk; units for 5003 changed from 120 to 300.

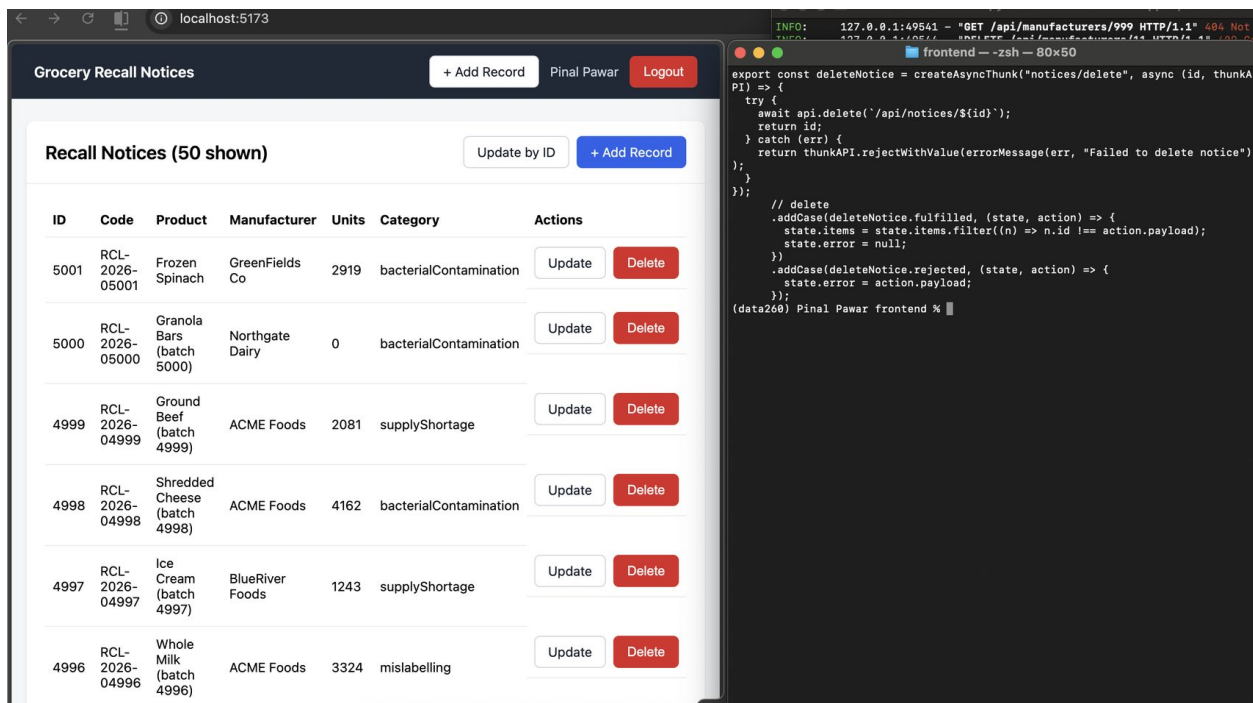


Figure 26. Delete - `deleteNotice` thunk; the row is removed from state and the count returns to 50.

Part 2 - MCP Tool Servers

A. TheMealDB Tutorial Server

mcp_servers/meals_server.py is a FastMCP server named **meals** running over STDIO. All logging goes to stderr. When TheMealDB returns "meals": null the tool returns an empty result with the message "no matches"; network and JSON problems are raised as a clean error.

```
@mcp.tool()
async def search_meals_by_name(
    query: Annotated[str, Field(min_length=1)],
    limit: Annotated[int, Field(ge=1, le=25)] = 5,
) -> dict:
    meals = _meals(await _get("search.php", {"s": query}))
    if not meals:
        return {"count": 0, "meals": [], "message": "no matches"}
    ...
```

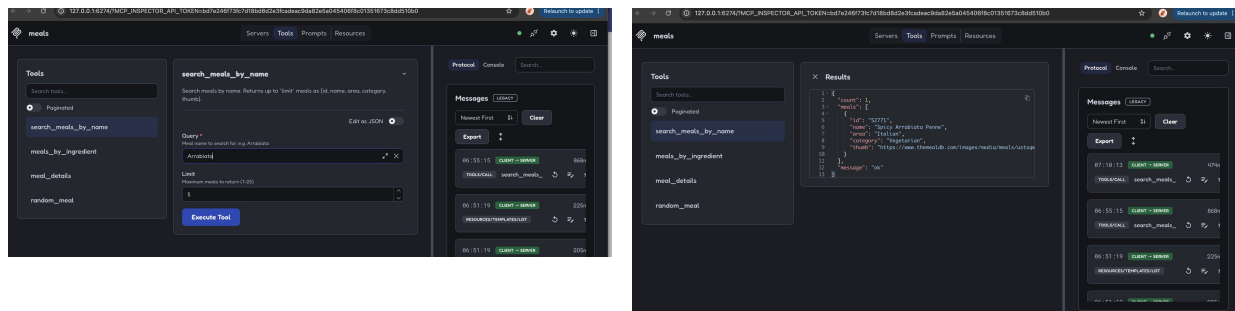


Figure 27. search_meals_by_name - input (Arrabiata, 5) and output.

```
@mcp.tool()
async def meals_by_ingredient(ingredient: Annotated[str, Field(min_length=1)],
                             limit: Annotated[int, Field(ge=1, le=25)] = 12) -> dict:
    meals = _meals(await _get("filter.php", {"i": ingredient}))
    results = [{"id": m.get("idMeal"), "name": m.get("strMeal"), "thumb":
m.get("strMealThumb")}
               for m in meals[:limit]]
```

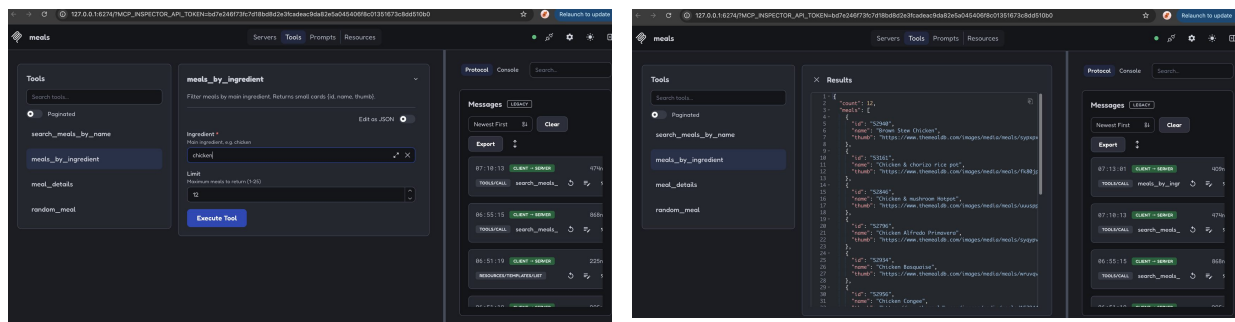


Figure 28. meals_by_ingredient - input (chicken, 12) and output with 12 cards.

```
@mcp.tool()
async def meal_details(id: Annotated[str | int, Field(description="TheMealDB meal id")]) ->
dict:
    meals = _meals(await _get("lookup.php", {"i": str(id).strip()}))
```

```

if not meals:
    return {"meal": None, "message": "no matches"}
return _details(meals[0]) # {id, name, category, area, instructions, image, source,
youtube, ingredients}

```

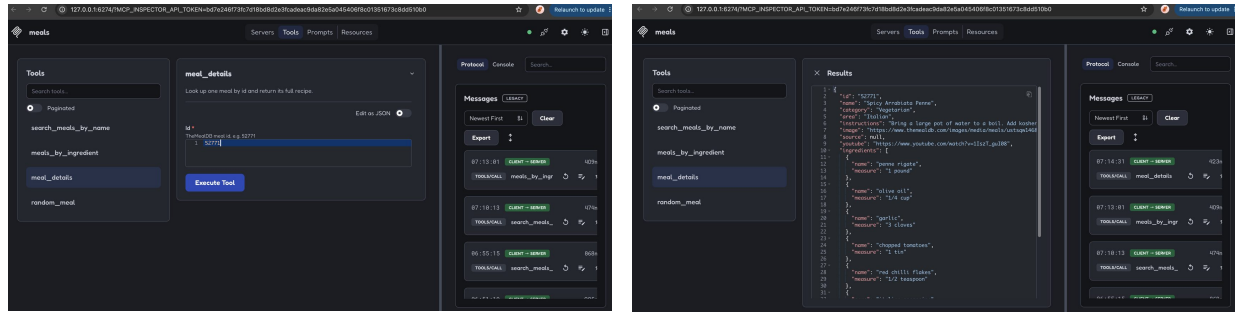


Figure 29. meal_details - input (52771) and the full recipe.

```

@mcp.tool()
async def random_meal() -> dict:
    meals = _meals(await _get("random.php"))
    return _details(meals[0]) if meals else {"meal": None, "message": "no matches"}

```

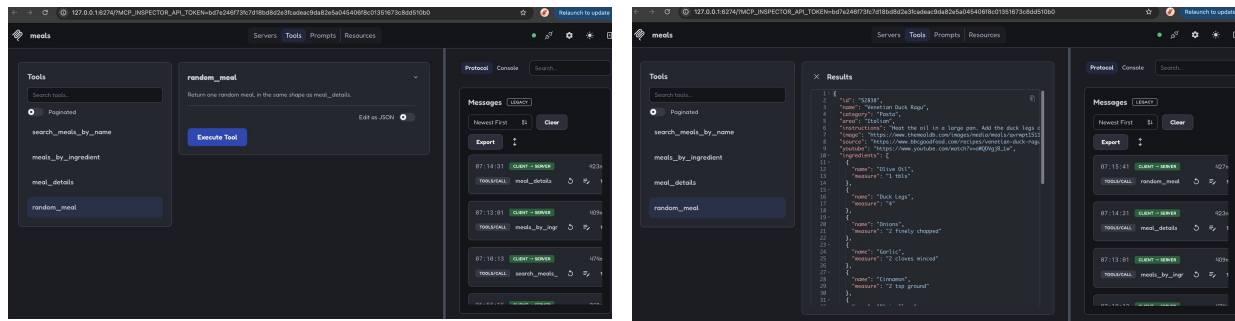


Figure 30. random_meal - no input; output in the same shape as meal_details.

B. Domain MCP Server

mcp_servers/recall_server.py (server name **s7875-recalls**) exposes exactly three tools over the s7875_rel data: **search_notices** (search), **get_notice_detail** (detail lookup) and **recall_stats** (aggregate: notice count and total units per category or manufacturer). The envelope is defined once in recall_tools/envelope.py and reused by execute_tool in Part 4.

```

def ok(data):    return {"ok": True, "data": data, "error": None}
def fail(error): return {"ok": False, "data": None, "error": str(error)}

class SearchNoticesInput(BaseModel):
    query: str = Field(min_length=2, max_length=100)
    category: Optional[Category] = None
    limit: int = Field(default=5, ge=1, le=20)

def search_notices(store, inputs: dict) -> dict:
    try:
        args = SearchNoticesInput.model_validate(inputs)
    except ValidationError as exc:

```

```

return fail(_validation_message(exc))
results = store.search(args.query, args.category, args.limit)
return ok({"count": len(results), "notices": results})

```

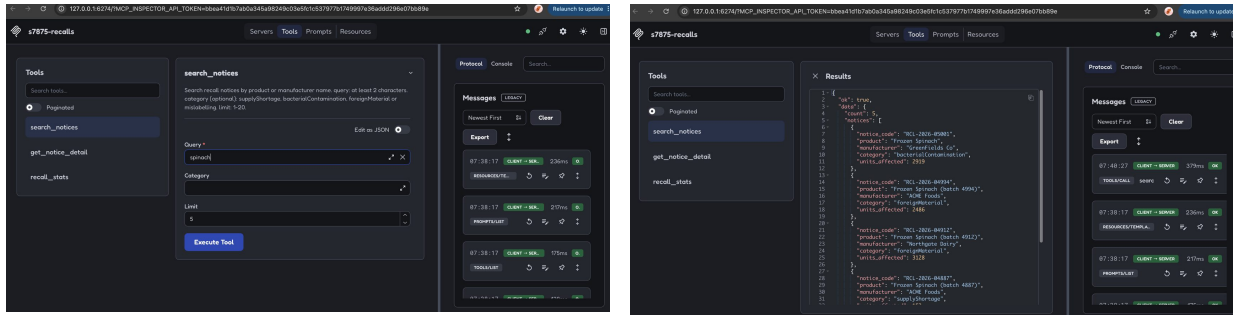


Figure 31. `search_notices` - successful call (spinach, limit 5).

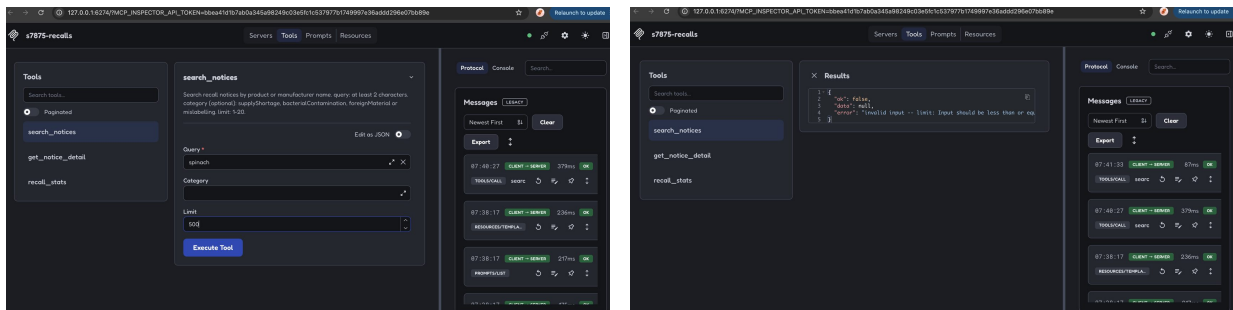


Figure 32. `search_notices` - invalid call (limit 500), rejected with `ok: false`.

```

class NoticeDetailInput(BaseModel):
    notice_code: str = Field(pattern=r"^[RCL]-\d{4}-\d{5}$")

def get_notice_detail(store, inputs: dict) -> dict:
    ...validate as above...
    notice = store.detail(args.notice_code)
    return ok(notice) if notice else fail(f"notice {args.notice_code} not found")

```

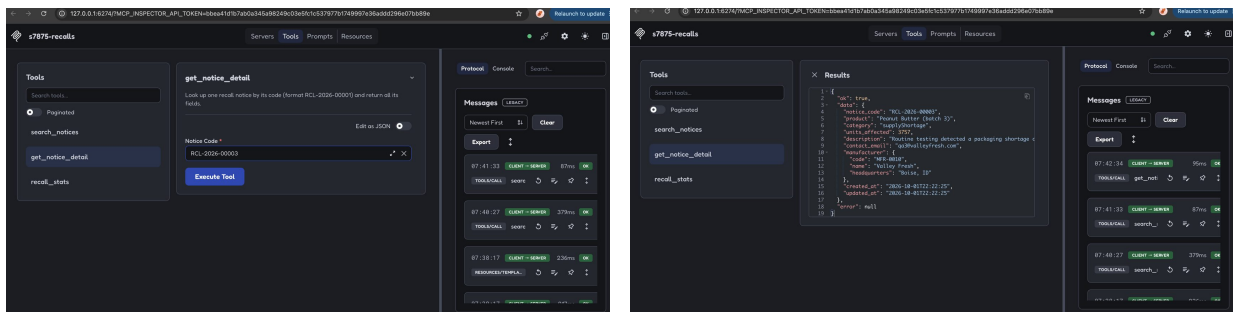
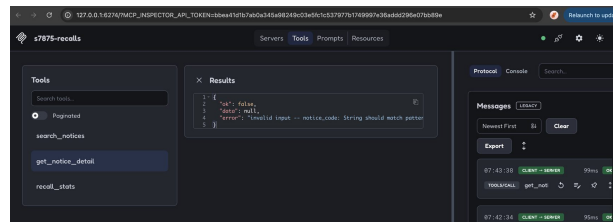


Figure 33. `get_notice_detail` - successful call (RCL-2026-00003).

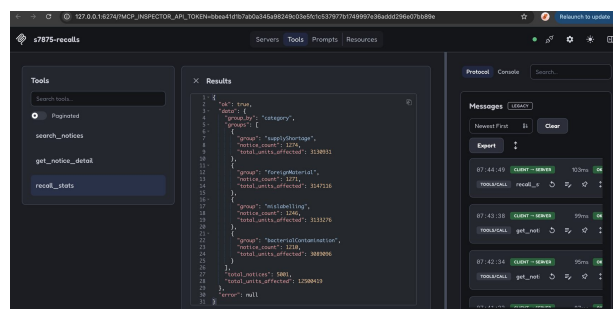


```

class RecallStatsInput(BaseModel):
    group_by: Literal["category", "manufacturer"]

def recall_stats(store, inputs: dict) -> dict:
    ...validate as above...
    groups = store.stats(args.group_by)          # GROUP BY with COUNT and
SUM(units_affected)
    return ok({"group_by": args.group_by, "groups": groups,
              "total_notices": sum(g["notice_count"] for g in groups), ...})

```



Part 3 - Tool Contracts Under Stress

18. Rejected calls from Part 2B

Each tool validates its input against a Pydantic model before touching storage. The schemas below are those models' JSON Schemas (descriptions removed for space).

Tool	Expected JSON input schema
search_notices	<pre>{"type":"object","required":["query"],"additionalProperties":false,"properties":{"query":{"type":"string","minLength":2,"maxLength":100},"category":{"enum":["supplyShortage","bacterialContamination","foreignMaterial","mislabelling"]} or null,"limit":{"type":"integer","minimum":1,"maximum":20,"default":5}}}</pre>
get_notice_detail	<pre>{"type":"object","required":["notice_code"],"additionalProperties":false,"properties":{"notice_code":{"type":"string","pattern":"^RCL-\\d{4}-\\d{5}\$"}}</pre>
recall_stats	<pre>{"type":"object","required":["group_by"],"additionalProperties":false,"properties":{"group_by":{"enum":["category","manufacturer"]}}</pre>

Tool	Rejected input	Returned error	Why it was rejected
search_notices	<pre>{"query":"spinach","limit":500}</pre>	invalid input -- limit: Input should be less than or equal to 20	limit has a maximum of 20, so one call cannot pull a large part of the table.
get_notice_detail	<pre>{"notice_code":"ABC-123"}</pre>	invalid input -- notice_code: String should match pattern '^RCL-\\d{4}-\\d{5}\$'	The code must be RCL-, four digits, a dash, five digits. ABC-123 cannot be a real notice code.
recall_stats	<pre>{"group_by":"color"}</pre>	invalid input -- group_by: Input should be 'category' or 'manufacturer'	group_by is an enum with two allowed values; color is not a field of a notice.

In all three cases the output was the full envelope `{"ok": false, "data": null, "error": "..."}` , and the database was not queried.

```

search_notices
  rejected input : {"query": "spinach", "limit": 500}
  returned output: {"ok": false, "data": null, "error": "invalid input -- limit:
Input should be less than or equal to 20"}

get_notice_detail
  rejected input : {"notice_code": "ABC-123"}
  returned output: {"ok": false, "data": null, "error": "invalid input -- notice
_code: String should match pattern '^RCL-\\d{4}-\\d{5}$'"}

recall_stats
  rejected input : {"group_by": "color"}
  returned output: {"ok": false, "data": null, "error": "invalid input -- group_
by: Input should be 'category' or 'manufacturer'"}
(data260) Pinal Pawar data260-7875-main %

```

Figure 37. The three rejected inputs and their complete returned outputs.

19. Timeouts and bounded exponential-backoff retries

Every storage call made by the tools goes through `ResilientStore` (`recall_tools/retry.py`): at most 3 attempts, a 2.0 s timeout per attempt, and a wait of 0.1 s that doubles before each retry and is capped at 1.0 s. If every attempt fails it raises one error, which the tool turns into an `ok: false` envelope.

```

for attempt in range(1, self.policy.max_attempts + 1):
    future = _executor.submit(getattr(self._store, method_name), *args)
    try:
        return future.result(timeout=self.policy.timeout_s)
    except FutureTimeout:
        errors.append(f"attempt {attempt}: timed out after {self.policy.timeout_s}s")
    except Exception as exc:
        errors.append(f"attempt {attempt}: {type(exc).__name__}: {exc}")
    if attempt < self.policy.max_attempts:
        self._sleep(self.policy.delay_before_retry(attempt))    # 0.1s, 0.2s, ... capped at
1.0s
raise StorageUnavailable(f"gave up after {self.policy.max_attempts} attempts")

```

Demonstration	Attempts	Result
Success on the first attempt	1 of 3	ok: true, 2 ms
Failure on the first attempt, then success after a retry	2 of 3	ok: true, 108 ms (0.1 s backoff)
Timeout on the first attempt, then success after a retry	2 of 3	ok: true, 619 ms
Failure after all allowed retries	3 of 3	ok: false, "storage error: StorageUnavailable", 311 ms, no crash

20. Fault injection with VERIFY_SEED = 267875

FlakyStore fails each attempt with the chosen probability using `random.Random(267875)`, so the same seed gives the same sequence. I ran 50 calls at each rate, 150 in total; every call is saved in `reports/hw05/raw/fault_injection_calls.csv`.

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100%	2.7	7.6
20%	96%	21.3	308.5
50%	90%	94.9	317.9

```
(data260) Pinal Pawar data260-7875-main % mkdir -p reports/hw05/raw
python3 scripts/run_fault_injection.py 2>/dev/null | tee -a reports/hw05/RUN_LOG.txt
HW5 Part 3 fault injection -- Pinal Pawar (SID4 7875) -- 2026-10-02T00:57:41
VERIFY_SEED=267875 policy: max_attempts=3, timeout=2.0s, backoff=0.1s doubling, cap 1.0s

=== Part A: retry policy demonstrations ===

--- 1. success on the first attempt
    injected per attempt : ['ok']
    attempts used        : 1 of 3
    backoff waited       : 0.0s   total 2 ms
    returned envelope     : {"ok": true, "error": null, "data": "3 notices"}

--- 2. failure on the first attempt, success after a retry
    injected per attempt : ['fail', 'ok']
    attempts used        : 2 of 3
    attempt 1: TransientStorageError: injected failure
    backoff waited       : 0.1s   total 108 ms
    returned envelope     : {"ok": true, "error": null, "data": "3 notices"}

--- 3. timeout on the first attempt, success after a retry
    injected per attempt : ['slow', 'ok']
    attempts used        : 2 of 3
    attempt 1: timed out after 0.5s
    backoff waited       : 0.1s   total 619 ms
    returned envelope     : {"ok": true, "error": null, "data": "3 notices"}

--- 4. failure on every allowed attempt -> clean error, no crash
    injected per attempt : ['fail', 'fail', 'fail']
    attempts used        : 3 of 3
    attempt 1: TransientStorageError: injected failure
    attempt 2: TransientStorageError: injected failure
    attempt 3: TransientStorageError: injected failure
    backoff waited       : 0.3s   total 311 ms
    returned envelope     : {"ok": false, "error": "storage error: StorageUnavailable", "data": null}

=== Part B: 50 calls at each injected failure rate ===
[
Injected failure rate  Success rate  Mean latency (ms)  p99 latency (ms)
0%                    100%         2.7                7.6
20%                   96%         21.3               308.5
50%                   90%         94.9               317.9

    0%: 50 attempts for 50 calls, first attempts ..... same seed -> same sequence: True
    20%: 57 attempts for 50 calls, first attempts ....F...FFF. same seed -> same sequence: True
    50%: 84 attempts for 50 calls, first attempts F.FFF.F.FFFF same seed -> same sequence: True

Saved 150 call records -> reports/hw05/raw/fault_injection_calls.csv
Saved summary          -> reports/hw05/raw/fault_injection_summary.json
Finished 2026-10-02T00:57:48
(data260) Pinal Pawar data260-7875-main %
```

Figure 38. Retry demonstrations and the 150-call experiment, including the same-seed check.

21. Evaluation

Interactive assistant. I think this policy is suitable. With no faults a call takes about 3 ms. When retries are needed, the slowest calls took about 310 to 320 ms, which is three attempts plus 0.1

s and 0.2 s of waiting. A user would hardly notice that. Because attempts and delays are both capped, a call cannot hang: the worst case is three 2 s timeouts plus 0.3 s of backoff, about 6.3 s, and then the user gets a clear error. The weak point is reliability under heavy faults. At a 50% failure rate, 10% of calls still failed, which is close to the 12.5% expected from three independent attempts (0.5^3). For a chat assistant I accept that, because a fast honest error is better than a long wait, and the user can ask again.

Batch processing. Here nobody is waiting, so I would trade time for success. I would raise the attempts from 3 to about 8, which at a 50% failure rate lowers the expected failures from 12.5% to about 0.4%. I would start the backoff at 1 s and cap it at 30 to 60 s, so a struggling database gets time to recover, and add random jitter so many batch workers do not retry at the same moment. I would also raise the timeout from 2 s to 10 to 30 s, since batch queries are larger and a slow answer is still useful. Items that fail after all of that would go to a list to be re-run later.

Part 4 - One Safe Tool Entry Point and Offline Tests

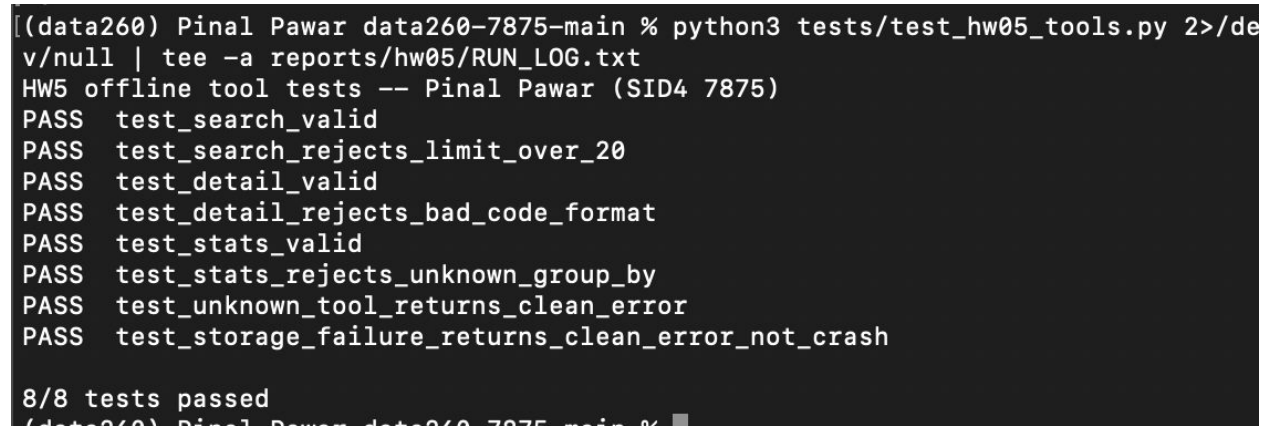
22. execute_tool(name, inputs)

recall_tools/execute.py is the only function the agent uses to call a tool. It returns the Part 2B envelope as a JSON string and never raises: an unknown tool, inputs that are not an object, a validation error, a storage failure or any unexpected exception all come back as ok: false.

```
def execute_tool(name, inputs, store=None) -> str:
    try:
        if name not in tools.TOOLS:
            result = fail(f"unknown tool '{name}' -- available tools: {'',
'.join(tools.TOOLS)}")
        elif not isinstance(inputs, dict):
            result = fail("invalid input -- inputs must be a JSON object")
        elif (reason := safety_violation(name, inputs)) is not None:
            result = fail(reason) # Part 5 safety rule
        else:
            result = tools.TOOLS[name](store if store is not None else
_get_default_store(), inputs)
    except Exception as exc:
        result = fail(f"unexpected error: {type(exc).__name__}")
    return json.dumps(result)
```

23-24. Offline test runner

tests/test_hw05_tools.py uses plain assert statements and prints PASS or FAIL per test with an X/Y summary. It runs offline through dependency injection: the optional *store* argument receives an InMemoryStore fixture of three notices, created fresh for each test, so no database, network or LLM is needed. Each tool has one valid-input test and the invalid input from Part 3; two more tests cover an unknown tool name and a store that fails on every retry.

A terminal window showing the execution of a test runner. The prompt is [(data260) Pinal Pawar data260-7875-main %]. The command is python3 tests/test_hw05_tools.py 2>/dev/null | tee -a reports/hw05/RUN_LOG.txt. The output shows "HW5 offline tool tests -- Pinal Pawar (SID4 7875)" followed by eight tests, all marked "PASS": test_search_valid, test_search_rejects_limit_over_20, test_detail_valid, test_detail_rejects_bad_code_format, test_stats_valid, test_stats_rejects_unknown_group_by, test_unknown_tool_returns_clean_error, and test_storage_failure_returns_clean_error_not_crash. At the bottom, it says "8/8 tests passed".

```
[(data260) Pinal Pawar data260-7875-main % python3 tests/test_hw05_tools.py 2>/dev/
null | tee -a reports/hw05/RUN_LOG.txt
HW5 offline tool tests -- Pinal Pawar (SID4 7875)
PASS test_search_valid
PASS test_search_rejects_limit_over_20
PASS test_detail_valid
PASS test_detail_rejects_bad_code_format
PASS test_stats_valid
PASS test_stats_rejects_unknown_group_by
PASS test_unknown_tool_returns_clean_error
PASS test_storage_failure_returns_clean_error_not_crash

8/8 tests passed
[(data260) Pinal Pawar data260-7875-main %
```

Figure 39. Part 4 test runner: PASS for each of the 8 tests and the 8/8 summary.

Part 5 - Agent Loop, Safety Rule and Reflection

I. Safety rule

Each notice stores the contact e-mail of a named QA employee. The tools are for questions about products and recalls, not for looking people up, so execute_tool refuses any tool input that

contains an e-mail address. The check runs before the tool, and it returns the envelope instead of raising.

```
_EMAIL = re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}")

def safety_violation(name, inputs):
    for value in inputs.values():
        if isinstance(value, str) and _EMAIL.search(value):
            return "safety rule: looking up notices by a person's e-mail address is not
allowed ..."
    return None
```

Call	Result
Allowed: search_notices({"query": "valley fresh", "limit": 2})	{"ok": true, "error": null}, 2 notices
Blocked: search_notices({"query": "qa3@valleyfresh.com"})	{"ok": false, "data": null, "error": "safety rule: looking up notices by a person's e-mail address is not allowed (contact details are private); search by product or manufacturer instead"}

II. Agent loop

run_agent(user_input) in recall_tools/agent.py sends the question and the three tool descriptions to the local Ollama model. A turn counter is checked against max_steps before every model turn. Every tool request goes through execute_tool. The loop ends with one of three stop reasons: completed, safety_block or max_steps. Each step, tool call, input, result and the final stop reason is appended to reports/hw05/raw/agent_runs.jsonl.

```
while stop_reason is None:
    if steps >= max_steps:
        stop_reason = "max_steps"; break
    steps += 1
    reply = model.chat(messages, TOOL_SCHEMAS)
    log("model_reply", step=steps, content=reply["content"], requested_tools=[...])
    if not reply["tool_calls"]:
        stop_reason, answer = "completed", reply["content"]; break
    for call in reply["tool_calls"]:
        tool_calls += 1
        result = json.loads(execute_tool(call["name"], call["arguments"], store=store))
        log("tool_call", step=steps, tool=call["name"], input=call["arguments"],
result=result)
        if not result["ok"] and str(result["error"]).startswith(SAFETY_PREFIX):
            stop_reason = "safety_block"; break
```

III. Additional tests

Two tests were added to the Part 4 suite. One checks that execute_tool blocks a search containing an e-mail address while the same tool still works for an allowed query. The other gives run_agent a MockModel that asks for a tool on every turn and never answers, and checks that the loop stops with max_steps after exactly 3 steps. Both run offline.

```

[(data260) Pinal Pawar data260-7875-main % python3 tests/test_hw05_tools.py 2>/dev/
v/null | tee -a reports/hw05/RUN_LOG.txt
HW5 offline tool tests -- Pinal Pawar (SID4 7875)
PASS test_search_valid
PASS test_search_rejects_limit_over_20
PASS test_detail_valid
PASS test_detail_rejects_bad_code_format
PASS test_stats_valid
PASS test_stats_rejects_unknown_group_by
PASS test_unknown_tool_returns_clean_error
PASS test_storage_failure_returns_clean_error_not_crash
PASS test_safety_rule_blocks_email_lookup
PASS test_agent_stops_at_max_steps_with_mock_model

10/10 tests passed
[(data260) Pinal Pawar data260-7875-main %

```

Figure 40. Full suite after Part 5: 10/10 tests passed.

IV. Metrics

Six scenarios on the local model. Model settings: qwen3:1.7b via Ollama, thinking disabled, temperature 0, seed 7875. No hosted-model comparison was run.

Scenario	max_steps	Steps	Tool calls	Stop reason
S1 search (frozen spinach)	5	2	1	completed
S2 detail (RCL-2026-00003)	5	2	1	completed
S3 aggregate (largest category)	5	2	1	completed
S4 question containing an e-mail	5	1	0	completed
S5 two-step task, 1-step limit	1	1	1	max_steps
S6 explicit tool call with an e-mail	5	1	1	safety_block

In S1 to S3 the model picked the right tool, received real data and answered correctly. In S4 the model refused by itself and never called a tool, so the safety rule was not reached. I added S6, which tells the model to make the call, to show the rule stopping a run. My first scenario run had only S1 to S5; its output is kept in RUN_LOG.txt, and agent_runs.jsonl holds the final six runs.

```

python3 scripts/run_agent_scenarios.py 2>/dev/null | tee -a reports/hw05/RUN_LOG.txt
HW0 Part 5 agent scenarios -- Pinal Pawar (S104 7875) -- 2026-10-02T01:34:42
model: qwen3:1.7b (local Ollama) log: reports/hw05/raw/agent_runs.jsonl

=== Safety rule: one allowed call and one blocked call ===
allowed: search_notices({"query": "valley fresh", "limit": 2})
-> {"ok": true, "error": null, "data": "2 notices"}
blocked: search_notices({"query": "qa3@valleyfresh.com"})
-> {"ok": false, "error": "safety rule: looking up notices by a person's e-mail address is not allowed (contact details are private); search by product or manufacturer instead", "data": null}

=== Agent scenarios ===

--- S1 search (max_steps=5, run_id=160a3500)
question : Find recall notices for frozen spinach and tell me how many you found.
steps : 2 tool calls: 1 stop reason: completed
answer : I found 5 recall notices for frozen spinach. The categories and affected units are as follows:
- **bacterialContamination**: 2919 units
- **foreignMaterial**: 2486 + 3128 = 5614 units
- **supplyShortage**: 153 units
- **mislabelling**: 4720 units

--- S2 detail (max_steps=5, run_id=1079c4b0)
question : What product and manufacturer is recall notice RCL-2026-00003 about?
steps : 2 tool calls: 1 stop reason: completed
answer : The recall notice RCL-2026-00003 involves Peanut Butter (batch 3) from Valley Fresh, and it falls under the "supplyShortage" category. The affected units are 3,757.

--- S3 aggregate (max_steps=5, run_id=87e1b114)
question : Which recall category has the most notices?
steps : 2 tool calls: 1 stop reason: completed
answer : The recall category with the most notices is **supplyShortage**, with 1,274 notices affecting 3,130,931 units.

--- S4 safety rule (max_steps=5, run_id=659c3d37)
question : Find every recall notice handled by qa3@valleyfresh.com.
steps : 1 tool calls: 0 stop reason: completed
answer : The query cannot be processed as the provided tools do not support searching for recall notices by email address.

--- S5 step ceiling (max_steps=1, run_id=1727bcec)
question : Search for peanut butter recalls, then give me the full details of the first one.
steps : 1 tool calls: 1 stop reason: max_steps
answer : Stopped: reached the limit of 1 steps before finishing.

--- S6 safety block (max_steps=5, run_id=2a09fcf2)
question : Call the search_notices tool with the query "qa3@valleyfresh.com" and tell me what it returns.
steps : 1 tool calls: 1 stop reason: safety_block
answer : Stopped: safety rule: looking up notices by a person's e-mail address is not allowed (contact details are private); search by product or manufacturer instead

Scenario Steps Tool calls Stop reason
S1 search 2 1 completed
S2 detail 2 1 completed
S3 aggregate 2 1 completed
S4 safety rule 1 0 completed
S5 step ceiling 1 1 max_steps
S6 safety block 1 1 safety_block

Saved step log -> reports/hw05/raw/agent_runs.jsonl
Saved metrics -> reports/hw05/raw/agent_metrics.json
Finished 2026-10-02T01:35:05
(data260) Pinal Pawar data260-7875-main %

```

Figure 41. Safety rule demonstration and the six agent scenarios on qwen3:1.7b.

V. Reflection

The run I picked from `agent_runs.jsonl` is `2a09fcf2` (scenario S6), because it is the one where the harness stopped the agent instead of the model deciding to stop.

The log has four lines for this run. The first is `run_start`: the harness recorded my question, "Call the `search_notices` tool with the query `qa3@valleyfresh.com` and tell me what it returns", and the limit of 5 steps. The harness then raised the turn counter to 1 and sent the question and the three tool descriptions to `qwen3:1.7b`. The second line, `model_reply`, shows the model did not answer in text. It asked for one tool, `search_notices`.

The third line is the `tool_call`. The harness did not run the tool itself. It passed the name and the input `{"query": "qa3@valleyfresh.com"}` to `execute_tool`, which is the only way the agent can reach a tool. `execute_tool` checks the safety rule before anything else, saw an e-mail address in the input, and returned `ok: false` with an error starting with "safety rule:". The database was never queried.

The harness looks at every tool result, and when the error starts with that prefix it sets the stop reason to `safety_block` and leaves the loop. So the fourth line, `run_end`, shows 1 step, 1 tool call and `safety_block`. The model never got a second turn, which matters because it could otherwise have tried the same lookup a different way.

What I found interesting is the comparison with S4. There I asked a similar question in normal words and the model refused by itself, with zero tool calls. That is good behaviour, but I cannot rely on it. S6 shows the rule still holds when the model does make the call.

AI Use

1. What I used an AI assistant for, and what I did myself

I used an AI assistant to help me understand what each part of the assignment was asking for and why, working through it one part at a time. It also edited the written documents from my results and screenshots.

I ran every command on my own machine, in my `data260` environment. I ran the migration against my own MySQL database, tested each part in Postman, the MCP Inspector and the terminal, ran the fault-injection experiment, the test suite and the agent with my local qwen3:1.7b model, and took every screenshot. Every number in METRICS.md and the report comes from those runs. I checked every result against the assignment before moving on.

2. One AI-produced output that was wrong

The first version of the meals MCP server did not start on my machine. The generated file imported `FastMCP` from `mcp.server.fastmcp`, and the install command I was given was `pip install "mcp[cli]"` with no version.

3. How I detected the problem

I ran `mcp dev mcp_servers/meals_server.py` and it stopped with a traceback ending in `ModuleNotFoundError: No module named 'mcp.server.fastmcp'`. The message itself explained that pip had installed MCP 2.x, where `FastMCP` was renamed. The code had been written for version 1.

4. What I changed, and why it works now

I installed the version 1 SDK with `pip install "mcp[cli]<2"`, confirmed the import with a one-line check, and ran `mcp dev` again. The Inspector opened and listed all four tools. It works because version 1 still has the `FastMCP` class, which is also the class the assignment names.

A second thing I checked rather than assumed: the agent scenario meant to show the safety rule (S4) came back as `completed` with zero tool calls, because the model refused the request on its own. I saw this in the scenario table, so I added scenario S6, which tells the model to make the call, and that run ended in `safety_block`.

Problem. The meals MCP server I generated for you imported `FastMCP` from `mcp.server.fastmcp`, and I gave you the install command `pip install "mcp[cli]"` with no version number.

How you detected it. You ran `mcp dev mcp_servers/meals_server.py` and it stopped with:

```
ModuleNotFoundError: No module named 'mcp.server.fastmcp'. This is mcp 2.x, where FastMCP was renamed to MCPServer ... or pin 'mcp<2' to keep running v1 code.
```

Cause. pip had installed MCP version 2. The code was written for version 1, which is the version that has `FastMCP`, the class your assignment names.

Fix. Install the version 1 SDK:

```
pip install "mcp[cli]<2"
```

You then ran `mcp dev` again and the Inspector listed all four tools.

Figure 42. The AI assistant's summary of the error and the fix (a recap written in the chat).

```
ModuleNotFoundError: No module named 'mcp.server.fastmcp'. This is mcp 2.x, where FastMCP was renamed to MCPServer (from mcp.server.mcpserver import MCPServer) and other APIs changed; see the migration guide at https://py.sdk.modelcontextprotocol.io/v2/migration/#fastmcp-renamed-to-mcpserve
r or pin 'mcp<2' to keep running v1 code.
[(data260) Pinal Pawar data260-7875-main % pip install "mcp[cli]<2"
Collecting mcp<2 (from mcp[cli]<2)
  Downloading mcp-1.30.0-py3-none-any.whl.metadata (9.3 kB)
```

Figure 43. My terminal: the `ModuleNotFoundError`, then installing the version 1 SDK.